Конашинская Ольга Александровна 090303-Пиб-о24

Лабораторная работа №

Вариант: № 10

Тема: **Применение паттернов проектирования при разработке объектно-ориентированных приложений на Python.**

Постановка задачи: Необходимо разработать приложение согласно индивидуальному варианту.

Основной целью является выбор и применение подходящих паттернов проектирования.

Работа состоит из трёх этапов:

Этап 1. Наивная реализация

Сначала необходимо реализовать выбранную функциональность без использования соответствующего паттерна.

Например:

```
if payment_type == "card":
    ...
elif payment_type == "cash":
    ...
elif payment_type == "bank":
    ...
```

Или:

```
if notification_type == "email":
    ...
elif notification_type == "sms":
    ...
```

Или:

```
if command == "add":
    ...
elif command == "delete":
    ...
```

Этап 2. Анализ

Необходимо определить проблему первоначального решения.

Для каждого паттерна необходимо ответить:

1. Какая проблема существует?
2. Почему текущая реализация плохо расширяется?
3. Какой паттерн подходит?
4. Какие классы входят в решение?
5. Какие зависимости между ними существуют?
6. Какие преимущества даёт паттерн?
7. Какие недостатки появляются?

Этап 3. Рефакторинг

Необходимо заменить наивную реализацию паттерном.

При этом:

- существующая бизнес-логика должна сохраниться;
- необходимо использовать абстракции;
- классы должны иметь понятную ответственность;
- новые реализации должны добавляться с минимальными изменениями существующего кода.

Обязательные паттерны

Для всех вариантов обязательны минимум три паттерна:

1. Repository
2. Strategy
3. Factory

Кроме того, в зависимости от варианта назначаются дополнительные паттерны.

Краткие теоретические сведения:

Понятие и структура паттерна проектирования

- Определение: Паттерн проектирования — это описание инвариантного (типового) решения многократно повторяющейся архитектурной проблемы в рамках определенного контекста.
- Природа сущности: Паттерн не является готовым программным кодом или библиотекой, а представляет собой концептуальную схему

взаимодействия объектов и классов, которая адаптируется под конкретную реализацию.

Классификация паттернов

В зависимости от решаемых задач паттерны разделяются на три категории:

- Порождающие (управление процессом инстанцирования объектов): *Factory, Abstract Factory, Builder, Prototype, Singleton*.
- Структурные (проектирование эффективных связей между компонентами): *Adapter, Decorator, Facade, Composite, Proxy*.
- Поведенческие (распределение обязанностей и алгоритмов взаимодействия): *Strategy, Observer, Command, State, Iterator, Template Method*.

Функциональный анализ рассматриваемых паттернов

- Factory (Фабрика): Локализует логику создания объектов, изолируя клиентский код от конкретных подклассов (устраняет громоздкие условные конструкции ветвления).
- Strategy (Стратегия): Инкапсулирует семейство взаимозаменяемых алгоритмов в отдельные классы, позволяя динамически изменять поведение основного сервиса без его модификации.
- Observer (Наблюдатель): Реализует механизм подписки, при котором изменение состояния одного объекта влечет автоматическое оповещение и обновление множества зависимых от него компонентов.
- Adapter (Адаптер): Обеспечивает совместную работу объектов с несовместимыми интерфейсами, выступая в роли транслятора вызовов к сторонним или legacy-системам.
- Repository (Репозиторий): Абстрагирует уровень доступа к данным, полностью скрывая детали физического хранения (БД, файлы, оперативная память) от бизнес-логики приложения.
- Command (Команда): Превращает операцию или запрос в самостоятельный объект, что позволяет унифицировать их запуск, вести историю действий, поддерживать отмену операций и формировать динамические очереди.

- Builder (Строитель): Обеспечивает пошаговое конструирование сложных объектов со множеством опциональных параметров с использованием цепочки вызовов (fluent interface).

Взаимосвязь паттернов и принципов SOLID

Паттерны проектирования выступают в качестве практического инструментария для соблюдения принципов SOLID:

- Factory (Фабрика): Обеспечивает соблюдение принципов открытости/закрытости (OCP) и единственной ответственности (SRP).
- Strategy (Стратегия): Реализует инверсию зависимостей (DIP) и позволяет расширять функционал без модификации кода (OCP).
- Observer (Наблюдатель): Направлен на минимизацию связанности (Loose Coupling) компонентов системы.
- Adapter (Адаптер): Позволяет интегрировать гетерогенные компоненты на основе принципа инверсии зависимостей (DIP).
- Repository (Репозиторий): Отвечает за декомпозицию ответственности (SRP) и изоляцию бизнес-логики от низкоуровневых реализаций (DIP).
- Command (Команда): Разделяет ответственность за инициацию и непосредственное исполнение запроса (SRP, OCP).
- Builder (Строитель): Снижает когнитивную сложность создания объектов и способствует соблюдению SRP.

Исходный код программы:

naive_version.py

```
class TicketApp:
    def __init__(self):
        self.tickets = []
        self.next_id = 1

    def process(self, command, passenger=None, train=None, car_type=None,
                tariff=None, ticket_id=None):
        if command == "buy":
            if car_type == "сидячий":
                base = 1000
            elif car_type == "купе":
                base = 2500
            elif car_type == "СВ":
                base = 5000
            else:
                print("Неизвестный тип вагона")
                return
```

```
            if tariff == "стандартный":
                price = base
            elif tariff == "льготный":
                price = base * 0.5
            elif tariff == "динамический":
                price = base * 1.3
            else:
                print("Неизвестный тариф")
                return

            ticket = {"id": self.next_id, "passenger": passenger,
                      "car": car_type, "price": price, "status": "оплачен"}
            self.tickets.append(ticket)
            self.next_id += 1
            print(f"Куплен билет #{ticket['id']} за {price} руб.")

        elif command == "return":
            for t in self.tickets:
                if t["id"] == ticket_id:
                    t["status"] = "возвращён"
                    print(f"Билет #{ticket_id} возвращён")
                    return
            print("Билет не найден")

        else:
            print("Неизвестная команда")


if __name__ == "__main__":
    app = TicketApp()
    app.process("buy", passenger="Иванов И.И.", car_type="купе",
tariff="стандартный")
    app.process("buy", passenger="Петров П.П.", car_type="СВ",
tariff="льготный")
    app.process("return", ticket_id=1)
```

models.py

```
from dataclasses import dataclass
from enum import Enum


class TicketStatus(Enum):
    RESERVED = "забронирован"
    PAID = "оплачен"
    USED = "использован"
    RETURNED = "возвращён"


@dataclass
class Ticket:
    id: int
    passenger: str
    train_number: str
    car_type: str
    base_price: float
    price: float = 0.0
    status: TicketStatus = TicketStatus.RESERVED
```

interfaces.py

```
from typing import Protocol
from models import Ticket
```

```
# Repository: интерфейс доступа к данным
class Repository(Protocol):
    def add(self, ticket: Ticket) -> None:
        ...

    def get(self, ticket_id: int) -> Ticket | None:
        ...

    def get_all(self) -> list[Ticket]:
        ...

    def delete(self, ticket_id: int) -> None:
        ...


# Strategy: интерфейс алгоритма расчёта цены
class PricingStrategy(Protocol):
    def calculate(self, base_price: float) -> float:
        ...


# Command: интерфейс операции-объекта
class Command(Protocol):
    def execute(self) -> None:
        ...
```

repositories.py

```
from models import Ticket


class InMemoryTicketRepository:
    def __init__(self) -> None:
        self._items: list[Ticket] = []

    def add(self, ticket: Ticket) -> None:
        self._items.append(ticket)

    def get(self, ticket_id: int) -> Ticket | None:
        for t in self._items:
            if t.id == ticket_id:
                return t
        return None

    def get_all(self) -> list[Ticket]:
        return list(self._items)

    def delete(self, ticket_id: int) -> None:
        self._items = [t for t in self._items if t.id != ticket_id]
```

factories.py

```
from models import Ticket


# Factory: создание билета нужного типа вагона сосредоточено в одном месте
class TicketFactory:
    PRICES = {
        "сидячий": 1000,
        "купе": 2500,
        "СВ": 5000,
    }
```

```
    @staticmethod
    def create(ticket_id: int, passenger: str, train_number: str, car_type:
str) -> Ticket:
        if car_type not in TicketFactory.PRICES:
            raise ValueError(f"Неизвестный тип вагона: {car_type}")
        base_price = TicketFactory.PRICES[car_type]
        return Ticket(ticket_id, passenger, train_number, car_type,
base_price)
```

strategies.py

```
class StandardPricing:
    def calculate(self, base_price: float) -> float:
        return base_price


class DiscountPricing:          # льготный тариф
    def calculate(self, base_price: float) -> float:
        return base_price * 0.5


class DynamicPricing:           # динамический тариф зависит от загрузки
    def __init__(self, load_factor: float = 0.3) -> None:
        self.load_factor = load_factor

    def calculate(self, base_price: float) -> float:
        return base_price * (1 + self.load_factor)
```

services.py

```
from models import Ticket, TicketStatus
from interfaces import PricingStrategy
from factories import TicketFactory


class TicketNotFoundError(Exception):
    pass


class TicketService:
    def __init__(self, repository) -> None:
        self.repository = repository      # Repository передаётся извне
        self._next_id = 1

    def create_ticket(self, passenger: str, train_number: str, car_type: str,
                      strategy: PricingStrategy) -> Ticket:
        ticket = TicketFactory.create(self._next_id, passenger, train_number,
car_type)
        ticket.price = strategy.calculate(ticket.base_price)   # Strategy
задаёт цену
        self._next_id += 1
        self.repository.add(ticket)
        return ticket

    def pay(self, ticket_id: int) -> None:
        ticket = self._get(ticket_id)
        ticket.status = TicketStatus.PAID

    def refund(self, ticket_id: int) -> None:
        ticket = self._get(ticket_id)
        ticket.status = TicketStatus.RETURNED

    def _get(self, ticket_id: int) -> Ticket:
        ticket = self.repository.get(ticket_id)
```

```
        if ticket is None:
            raise TicketNotFoundError(f"Билет #{ticket_id} не найден")
        return ticket
```

commands.py

```
from models import Ticket
from interfaces import PricingStrategy
from services import TicketService


# Command: каждая операция оформлена отдельным объектом с методом execute()
class BuyTicketCommand:
    def __init__(self, service: TicketService, passenger: str, train_number:
str,
                 car_type: str, strategy: PricingStrategy) -> None:
        self.service = service
        self.passenger = passenger
        self.train_number = train_number
        self.car_type = car_type
        self.strategy = strategy
        self.ticket: Ticket | None = None

    def execute(self) -> None:
        self.ticket = self.service.create_ticket(
            self.passenger, self.train_number, self.car_type, self.strategy)
        self.service.pay(self.ticket.id)


class ReturnTicketCommand:
    def __init__(self, service: TicketService, ticket_id: int) -> None:
        self.service = service
        self.ticket_id = ticket_id

    def execute(self) -> None:
        self.service.refund(self.ticket_id)
```

main . py

```
from repositories import InMemoryTicketRepository
from services import TicketService, TicketNotFoundError
from strategies import StandardPricing, DiscountPricing, DynamicPricing
from commands import BuyTicketCommand, ReturnTicketCommand

STRATEGIES = {
    "1": StandardPricing(),
    "2": DiscountPricing(),
    "3": DynamicPricing(),
}
CAR_TYPES = {"1": "сидячий", "2": "купе", "3": "СВ"}


def main() -> None:
    repository = InMemoryTicketRepository()
    service = TicketService(repository)
    history = []            # история выполненных команд (Command)
```

```
    while True:
        print("===== ПРОДАЖА ЖЕЛЕЗНОДОРОЖНЫХ БИЛЕТОВ =====",
              "1 - Купить билет",
              "2 - Вернуть билет",
              "3 - Показать все билеты",
              "4 - История операций",
              "0 - Выход", sep="\n")
        choice = input("Выберите пункт меню: ")

        if choice == "1":
            try:
                passenger = input("ФИО пассажира: ")
                train_number = input("Номер поезда: ")
                print("Тип вагона: 1 - сидячий, 2 - купе, 3 - СВ")
                car_type = CAR_TYPES.get(input("Выберите: "))
                print("Тариф: 1 - стандартный, 2 - льготный, 3 -
динамический")
                strategy = STRATEGIES.get(input("Выберите: "))
                if car_type is None or strategy is None:
                    print("Некорректный выбор.")
                    continue
                command = BuyTicketCommand(service, passenger, train_number,
car_type, strategy)
                command.execute()
                history.append(f"Куплен билет #{command.ticket.id} за
{command.ticket.price} руб.")
                print(f"Куплен билет #{command.ticket.id} за
{command.ticket.price} руб.")
            except ValueError as e:
                print("Ошибка:", e)

        elif choice == "2":
            try:
                ticket_id = int(input("ID билета: "))
                command = ReturnTicketCommand(service, ticket_id)
                command.execute()
                history.append(f"Возвращён билет #{ticket_id}")
                print("Билет возвращён.")
            except TicketNotFoundError as e:
                print("Ошибка:", e)
            except ValueError:
                print("Ошибка: введено не число.")

        elif choice == "3":
            tickets = repository.get_all()
            if not tickets:
                print("Билетов нет.")
            for t in tickets:
                print(f"#{t.id}: {t.passenger}, поезд {t.train_number}, "
                      f"{t.car_type}, {t.price} руб. — {t.status.value}")

        elif choice == "4":
            if not history:
                print("Операций ещё не было.")
            for i, record in enumerate(history, 1):
                print(f"{i}. {record}")
```

```
        elif choice == "0":
            print("Программа завершена!")
            break
        else:
            print("Некорректный ввод.")


if __name__ == "__main__":
    main()
```

tests.py

```
import unittest
from models import TicketStatus
from factories import TicketFactory
from strategies import StandardPricing, DiscountPricing, DynamicPricing
from repositories import InMemoryTicketRepository
from services import TicketService, TicketNotFoundError
from commands import BuyTicketCommand, ReturnTicketCommand


class TicketTests(unittest.TestCase):

    def test_factory_price(self):
        ticket = TicketFactory.create(1, "Иванов И.И.", "001A", "купе")
        self.assertEqual(ticket.base_price, 2500)

    def test_factory_unknown_type(self):
        with self.assertRaises(ValueError):
            TicketFactory.create(1, "Иванов И.И.", "001A", "люкс")

    def test_standard_strategy(self):
        self.assertEqual(StandardPricing().calculate(1000), 1000)

    def test_discount_strategy(self):
        self.assertEqual(DiscountPricing().calculate(1000), 500)

    def test_dynamic_strategy(self):
        self.assertAlmostEqual(DynamicPricing(0.3).calculate(1000), 1300)

    def test_service_create_and_pay(self):
        service = TicketService(InMemoryTicketRepository())
        ticket = service.create_ticket("Иванов И.И.", "001A", "сидячий",
StandardPricing())
        service.pay(ticket.id)
        self.assertEqual(ticket.status, TicketStatus.PAID)

    def test_refund_missing(self):
        service = TicketService(InMemoryTicketRepository())
        with self.assertRaises(TicketNotFoundError):
            service.refund(999)

    def test_buy_command(self):
        service = TicketService(InMemoryTicketRepository())
        command = BuyTicketCommand(service, "Петров П.П.", "055B", "СВ",
DiscountPricing())
        command.execute()
        self.assertEqual(command.ticket.price, 2500)
        self.assertEqual(command.ticket.status, TicketStatus.PAID)

    def test_return_command(self):
```

```
        service = TicketService(InMemoryTicketRepository())
        buy = BuyTicketCommand(service, "Сидоров С.С.", "001А", "купе",
StandardPricing())
        buy.execute()
        ReturnTicketCommand(service, buy.ticket.id).execute()
        self.assertEqual(buy.ticket.status, TicketStatus.RETURNED)


if __name__ == "__main__":
    unittest.main()
```

```
===== ПРОДАЖА ЖЕЛЕЗНОДОРОЖНЫХ БИЛЕТОВ =====
1 - Купить билет
2 - Вернуть билет
3 - Показать все билеты
4 - История операций
0 - Выход
Выберите пункт меню: 1
ФИО пассажира: ФИО
Номер поезда: 1
Тип вагона: 1 - сидячий, 2 - купе, 3 - СВ
Выберите: 1
Тариф: 1 - стандартный, 2 - льготный, 3 - динамический
Выберите: 2
Куплен билет #1 за 500.0 руб.
```

```
Выберите пункт меню: 2
ID билета: 1
Билет возвращён.
```
ы

```
Выберите пункт меню: 3
#1: ФИО, поезд 1, сидячий, 500.0 руб. — возвращён
```

```
Выберите пункт меню: 4
1. Куплен билет #1 за 500.0 руб.
2. Возвращён билет #1
```